Spring JDBC — Quick Revision Notes

1. Problem with Raw JDBC

Raw JDBC needs a lot of repeated code — open connection, create statement, close statement, close connection, catch exceptions. Only SQL + parameters + result handling is actually useful; rest is boilerplate.

2. Template Idea Behind Spring JDBC

Spring separates "fixed steps" (connection, execution, cleanup, exceptions) from "changing steps" (SQL, parameters, mapping). Spring handles fixed steps, developer handles changing steps. Main class for this = `JdbcTemplate`.

3. DataSource

`DataSource` is an interface whose job is to give database `Connection` objects. It's just a **connection provider**, not a connection itself.

3.1 DriverManager vs DataSource

`DriverManager` = old way, repository itself knows URL/username/password. `DataSource` = better way, connection details are injected from outside, repo doesn't need to know how connection was made.

4. Connection Pooling

Instead of creating a new physical connection every time (expensive), a pool keeps ready-made connections. App borrows a connection, uses it, and "closes" it (which actually just returns it to the pool).

4.3 What if all connections are busy?

New request waits until a connection is free. If none becomes free within timeout, an exception is thrown. This protects DB from too many connections at once.

4.4 Why limit connections?

Every connection uses memory, sockets, sessions etc. Too many connections can overload the database. Pooling gives better performance + stability.

5. DataSource ≠ Connection Pool

`DataSource` is just an interface — some implementations pool connections, some don't. `HikariDataSource` is an example of a `DataSource` that is backed by a pool (HikariCP). Repository only depends on `DataSource` interface, so implementation can be swapped anytime (dependency inversion).

6. HikariCP

HikariCP is the connection pool Spring Boot uses by default (comes with `spring-boot-starter-jdbc`). It manages pool size, timeouts, idle connections, etc.

7. Creating a Spring JDBC Project

Need `spring-boot-starter-jdbc` + DB driver (like MySQL connector) as dependencies. DB details go in `application.properties` under `spring.datasource.*`. HikariCP settings go under `spring.datasource.hikari.*`.

8. How Spring Boot Creates DataSource Bean

Spring Boot automatically reads your properties, picks HikariCP (if available), creates the `DataSource` bean itself. You usually don't need to create it manually.

9. Introducing JdbcTemplate

`JdbcTemplate` is Spring's main class to do JDBC work easily. Main methods: `update()` (for insert/update/delete), `query()` (for multiple rows), `queryForObject()` (for exactly one row).

9.1 JdbcTemplate + DataSource relation

`JdbcTemplate` itself can't create connections — it asks `DataSource` for one, uses it, then gives it back. Spring Boot auto-creates this bean too.

9.2 What JdbcTemplate handles internally

For update: get connection → create statement → bind params → execute → get row count → close everything. For select: same steps + also loop through ResultSet rows and call RowMapper for each row.

10. Project Structure

Normal layered structure: `Controller` → `Service` → `Repository` (+ `RowMapper`) → `Model`. Simple `students` table example with id, name, email, age.

11. Create (Insert) Operation

Use `jdbcTemplate.update(sql, params...)` for INSERT. Params bind in same order as `?` in query — order must match, else wrong data goes in wrong column.

11.2 Affected Row Count

`update()` returns number of rows changed (int). For single insert, you expect it to be `1`.

12. Read Operations

Select queries return rows, and rows must be converted into Java objects — this conversion job is done by `RowMapper`.

12.1 RowMapper

`RowMapper<T>` interface converts **one row → one Java object**. Its method `mapRow()` is called once per row automatically by Spring.

12.3 query() method

Use `query()` when 0, 1, or many rows can come. Returns a `List<T>` — empty list if nothing found (never null).

12.4 Reusing Mapper

Since RowMapper has no shared state (just reads current row), you can create it once and reuse it for every query instead of creating new each time.

12.5 Lambda RowMapper

Since `RowMapper` is a functional interface, you can write it as a lambda instead of a full class — shorter for simple mappings.

13. queryForObject()

Use when you expect **exactly one row** (like fetch by primary key).

- 0 rows → throws `EmptyResultDataAccessException`
- more than 1 row → throws `IncorrectResultSizeDataAccessException`
- Does NOT return null on no result — it throws exception.

13.2 Returning Optional

To handle "no student found" nicely, wrap `queryForObject()` in try-catch: catch only `EmptyResultDataAccessException` and return `Optional.empty()`. Don't catch every exception — DB failures should still throw, only "not found" should become empty optional.

13.3 Alternative using query()

You can also use `query()` (returns list) + `stream().findFirst()` to get Optional, avoiding exception-based logic. Trade-off: `queryForObject()` is clearer about "expecting one row".

14. Update Operation

Same as insert but with `UPDATE ... SET ... WHERE id = ?` — params order matters (set values first, then id last for WHERE). Returns 1 if updated, 0 if no match found.

15. Delete Operation

`DELETE FROM students WHERE id = ?` using `jdbcTemplate.update(sql, id)`. Same pattern — insert/update/delete sab `update()` method use karte hain kyunki teeno DB state change karte hain aur row count return karte hain.

16. BeanPropertyRowMapper

Spring's ready-made mapper — auto matches DB column names to Java bean properties (like `email` column → `email` field, `first_name` → `firstName`). Needs no-arg constructor + setters in class.

- Good for: simple tables, quick prototypes.
- Bad for: complex joins, when you need explicit control (custom RowMapper is better then).

17. Exception Translation

Raw JDBC throws `SQLException` (checked exception — must catch/declare it, very generic, vendor-specific codes). Spring catches it internally and converts to its own **unchecked** exception family called `DataAccessException`. So repository code doesn't need try-catch everywhere.

18. Common Spring Exceptions

- **DuplicateKeyException** — unique/primary key violated (e.g. duplicate email).
- **DataIntegrityViolationException** — broader constraint violation (foreign key, not-null, check constraint).
- **BadSqlGrammarException** — wrong SQL syntax or wrong column name.
- **CannotGetJdbcConnectionException** — connection couldn't be obtained (DB down, wrong credentials, pool timeout).
- **EmptyResultDataAccessException** — expected a row but got none (from queryForObject).
- **IncorrectResultSizeDataAccessException** — got wrong number of rows than expected.

19. Why Exception Translation Helps

If you switch database (MySQL → PostgreSQL), vendor-specific error codes change. But Spring's exceptions (like `DuplicateKeyException`) stay the same regardless of DB — so your code doesn't need to change. Improves portability.

20. Repository Abstraction

Controller/Service layers don't need to know if repository internally uses raw JDBC, Spring JDBC, JPA or Hibernate. This detail stays hidden inside repository layer — easy to change later without breaking other layers.

21. Raw JDBC vs Spring JDBC (comparison)

Spring JDBC keeps full SQL control (same as raw) but automates connection handling, statement creation, result iteration, resource cleanup, and gives unchecked exceptions + built-in pooling support. Much less boilerplate overall.

22. Complete Execution Flow

Insert flow: Repository → `update()` → get connection from pool → create statement → bind params → execute → get row count → close statement → return connection to pool → row count sent back. **Select flow:** Repository → `query()` → get connection → execute → get ResultSet → RowMapper converts each row → collect into List → close everything → connection back to pool → List returned.

23. Key Takeaways (super short revision)

1. Spring JDBC uses JDBC internally, doesn't remove SQL.
2. `DataSource` = connection provider (interface).
3. Pooling is optional in DataSource, HikariCP is Boot's default pool.
4. `JdbcTemplate` = handles repetitive JDBC steps.
5. `update()` → insert/update/delete. `query()` → many rows. `queryForObject()` → exactly one row.
6. `RowMapper` → converts 1 row to 1 object. `BeanPropertyRowMapper` → automatic but less control.
7. `SQLException` (checked) → translated into `DataAccessException` (unchecked) family.
8. "Not found" ≠ "DB error" — handle differently.